

EncryptEdge Labs

Junior Cloud Security Engineer Internship Programme

Junior Cloud Security Engineer Internship Task Report

Field	Value
Intern Full Name	Sukhmandeep Kaur
Intern Email	[REDACTED]
Cohort	4 cohort 2026
Task ID	CSE-AWS-L-013-04
Task Title	Cloud Credential Types, Secret Lifecycle Management, and Exposure Risks
Skill Domain	AWS Cloud Security / Secrets Management
Difficulty Level	Intermediate
CPE Credits	4 CPE Credits (task metadata also states 8 credits in CPD/CPE line; verify with LMS)
Guiding Framework(s)	MITRE ATT&CK, NIST SP 800-53, CIS Control 6, NIST CSF, ISO 27001/27005, OWASP SAMM, SFIA, NICE
Submission Date	29 August 2026

All activities documented in this report are limited to authorised training/lab environments. No real credentials are reproduced.

1.0 Task Overview

Field	Task-specific value
Task ID	CSE-AWS-L-013-04
Task Title	Cloud Credential Types, Secret Lifecycle Management, and Exposure Risks
Skill Domain	AWS Cloud Security / Secrets Management
Difficulty	Intermediate
Duration	12 Hours
CPE Credits	4 CPE Credits
Cloud Platform(s)	AWS CloudShell / AWS sandbox; CloudGoat; PwnedLabs; optional TryHackMe
AWS Region(s) Used	us-east-2 observed in AWS CLI configuration
Primary Tools	AWS CLI v2.36.31, CloudGoat, Terraform v1.16.0, TruffleHog v3.97.1, Git, ChatGPT; GitLeaks planned/used when report is available
Target Environment	AWS sandbox identity; CloudGoat; local synthetic Git repository; PwnedLabs hosted labs
MITRE ATT&CK IDs	T1552.001 Credentials in Files; T1555 Credentials from Password Stores; T1078 Valid Accounts
Guiding Frameworks	NIST SP 800-53 AC-2; CIS Control 6; ISO 27005; NIST CSF PR.AC-1; OWASP SAMM
Regulatory Alignment	ISO 27001 A.9 Access Control; A.12 Operations Security; SFIA SCTY3/CLDS3; NICE OPM622

1.1 Introduction

This task examined how AWS credentials are issued, used, stored, discovered, rotated, and revoked, with emphasis on the risk created when long-lived secrets are embedded in source repositories or other developer artefacts. The work combined AWS identity verification, an authorised CloudGoat setup, synthetic repository secret scanning, lifecycle risk analysis, and hosted-lab reinforcement.

1.2 Objectives & Learning Outcomes

- Identify AWS credential types (IAM user keys, STS tokens, instance roles) - reviewed credential characteristics and verified the active AWS identity with STS.
- Map the secret lifecycle from creation to revocation and identify weak points - produced a lifecycle diagram and exposure matrix in Phase D.
- Detect exposed secrets in code repositories using TruffleHog/GitLeaks - TruffleHog was run against a controlled synthetic repository; GitLeaks report remains subject to evidence confirmation.
- Evaluate risk severity using a threat model matrix - scored repository exposure, long-lived credentials, and weak rotation/revocation scenarios.
- Recommend secure secret management practices - mapped mitigations to AWS Secrets Manager/Parameter Store, KMS, least privilege, rotation, revocation, and CI/CD scanning.

1.3 Scope & Legal Authorisation

All activity was confined to authorised training environments and synthetic data. Environments used included the user's AWS sandbox/CloudShell session, CloudGoat, a local synthetic Git repository, and the named PwnedLabs/TryHackMe training platforms. No external production AWS accounts, third-party systems, or real customer data were targeted. AI was used for explanation, summarisation, risk-language refinement, and report drafting; technical CLI outputs and hosted-lab completion evidence must come from the intern's own execution.

1.4 MITRE ATT&CK Cloud Threat Mapping

Technique	Name	Relevance
T1552.001	Unsecured Credentials: Credentials In Files	Hardcoded or committed credentials may be recovered from repositories/configuration files.
T1555	Credentials from Password Stores	Relates to the broader risk of retrieving stored secrets from developer/runtime stores.
T1078	Valid Accounts	Exposed valid credentials can be used to authenticate as legitimate identities.

2.0 Environment & Tool Setup

2.1 Cloud Architecture Overview

Component	Type / Service	Role in Task	Config Notes
AWS CloudShell	AWS managed shell	CLI execution and evidence capture	Region observed: us-east-2
AWS IAM / STS	AWS identity services	Credential identity verification	aws sts get-caller-identity used
CloudGoat	Vulnerable cloud lab	Controlled IAM scenario deployment	Configured to use cloudgoat-lab profile; troubleshooting required
Synthetic Git repo	Local Git repository	Safe secret-scanning target	Contained deliberately fake training values only
PwnedLabs	Hosted training platform	Secret exposure detection practice	Two required labs; screenshots required
TryHackMe Secure GitOps	Hosted training platform	Optional CI/CD secret handling practice	Optional evidence

2.2 AWS CLI & Tool Configuration

Tool	Version / Status	Installation / Configuration	Purpose
AWS CLI	v2.36.31	Available in AWS CloudShell	Identity and IAM verification
Terraform	v1.16.0	Installed in CloudShell	CloudGoat infrastructure provisioning
CloudGoat	Repository module	Git clone; invoked with python3 -m cloudgoat.cloudgoat	Authorised vulnerable scenario
TruffleHog	v3.97.1	Standalone binary in ~/.local/bin	Repository secret scanning
GitLeaks	Evidence not yet supplied	Standalone binary workflow	Second scanner / comparison
ChatGPT	AI assistant	Interactive analysis	Concept explanation, scan interpretation, report drafting

Key configuration / verification commands:

```
aws --version
aws configure list
aws sts get-caller-identity
terraform --version
python3 -m cloudgoat.cloudgoat --help
trufflehog --version
```

2.3 AI Tool Usage Declaration

AI Tool	Used For	What Was NOT Done with AI
ChatGPT	Explaining credential types; troubleshooting; summarising scanner behaviour; structuring lifecycle/risk analysis; drafting report language	AI did not execute AWS commands, complete hosted labs, or create authentic lab-completion screenshots. Technical evidence must originate from the intern's own environment.

3.0 Methodology - Phase Execution

Phase A: Theory & Conceptual Foundation

Phase Objective: Understand the types of AWS credentials and their security implications. Estimated Time: 60 minutes.

Actions Taken: Reviewed IAM user access keys, temporary STS session credentials, and instance-role credentials. Verified the active AWS identity using AWS STS and compared long-lived credentials with short-lived session credentials.

```
aws sts get-caller-identity
# Returned an IAM user ARN for "██████████" (account value redacted in report).
```

Key Observations: IAM access keys are long-lived until rotated or disabled, increasing exposure duration if leaked. STS and instance-role credentials are temporary and reduce persistence, although they still require least privilege and protection during their validity window.

Evidence Produced: phaseA_identity.png - credential-flow diagram .

Phase B: Cloud Environment Setup & Target Preparation

Phase Objective: Deploy a controlled vulnerable environment using CloudGoat. Estimated Time: 90 minutes.

Actions Taken: Used an existing CloudGoat clone, confirmed Terraform availability, corrected Python module invocation, configured the CloudGoat AWS profile, and configured the IP whitelist. Troubleshooting included IAM ListUsers denial for the [REDACTED] identity, CloudShell storage exhaustion during provider installation, and missing Terraform variables before the CloudGoat AWS profile was configured.

```
python3 -m cloudgoat.cloudgoat --help
python3 -m cloudgoat.cloudgoat config aws
python3 -m cloudgoat.cloudgoat config whitelist
aws iam list-users
```

Key Observations: The current [REDACTED] identity did not have iam:ListUsers permission. CloudGoat required a configured AWS profile and sufficient CloudShell storage. These setup issues demonstrated how identity permissions and local execution constraints affect lab deployment.

Evidence Produced: phaseB_setup.png - terminal evidence showing CloudGoat configuration/setup state.

Phase C: Core Execution - Secret Discovery

Phase Objective: Identify exposed secrets within simulated repositories. Estimated Time: 150 minutes.

Actions Taken: The full SecLists clone exceeded CloudShell storage, so a small controlled Git repository was created with synthetic training-only credential-like values. TruffleHog v3.97.1 scanned the repository with verification disabled and output was redirected to trufflehog_output.txt.

```
trufflehog git file://$HOME/secret-lab --no-verification > ~/trufflehog_output.txt 2>&1
```

Key Observations: TruffleHog completed successfully and reported 0 verified and 0 unverified secrets in the captured scan. This demonstrates that secret-like strings are not automatically findings; detector format, entropy, supported providers, and validation logic affect results. A GitLeaks report is required for the planned cross-tool comparison, but its actual contents were not supplied for this report and therefore no GitLeaks finding count is asserted.

Evidence Produced: trufflehog_output.txt; gitleaks_report.json phaseC_summary.png.

Phase D: Independent Extension - Secret Lifecycle Analysis

Phase Objective: Map the lifecycle of discovered secrets and propose mitigations. Estimated Time: 180 minutes.

Actions Taken: Built a lifecycle model covering creation, storage, use, rotation, and revocation. Because no live/valid credential was confirmed by the supplied scan evidence, the analysis uses the synthetic repository credential patterns as representative exposure scenarios rather than claiming a real leak. Risks were scored by likelihood and impact and mapped to preventative and detective controls.

Key Observations: The highest-risk condition is a long-lived credential committed to version control with no automatic rotation. Repository deletion alone is insufficient because Git history and downstream clones may retain the secret; revocation/rotation is required. Temporary role credentials materially reduce persistence but do not eliminate misuse during the active session.

Evidence Produced: phaseD_lifecycle.pdf - lifecycle diagram, risk matrix, and recommendations.

Asset / Resource	Threat / Risk	MITRE	Likelihood	Impact	Score	Rating
IAM access key in Git	Long-lived credential exposed in	T1552.001 / T1078	4	5	20	Critical

	repository/history					
API/automation token	Hardcoded token reused without managed rotation	T1552.001	4	4	16	Critical
Database password/config secret	Plaintext secret in configuration file	T1552.001	3	4	12	High
Temporary STS / role credential	Session token captured during active validity window	T1078	2	4	8	Medium

Phase E: Hosted Lab Integration

Phase Objective: Reinforce learning through external labs on secret exposure detection. Estimated Time: 180 minutes.

Required hosted exercises: PwnedLabs - Hunt for Secrets in Git Repos; PwnedLabs - Uncover Secrets in CodeCommit and Docker. Optional: TryHackMe - Secure GitOps.

Actions Taken / Status: The task brief requires these labs, but no completion screenshots were supplied with this report request. To avoid fabricating evidence, the report records the labs as pending evidence confirmation. After completion, insert the intern's own screenshots and briefly document the secret-discovery technique, affected storage location, and remediation lesson from each lab.

Evidence Produced: phaseE_lab1.png (pending); phaseE_lab2.png (pending); phaseE_thm.png (optional/pending).

Phase F: Documentation & Reporting

Phase Objective: Consolidate all findings into a professional report suitable for executive review.

Actions Taken: Consolidated task metadata, phase execution, observed tool behaviour, lifecycle analysis, risk ratings, control gaps, recommendations, and evidence references into the EncryptEdge Labs report structure. Sensitive identifiers and credential values are intentionally excluded.

Key Observations: Evidence quality depends on traceability between the claim, the actual CLI/tool output, and the referenced screenshot/report. Unverified hosted-lab or GitLeaks results are marked pending rather than represented as completed.

Evidence Produced: CSE_CSE-AWS-L-013-04_S.K_20260829

4.0 Findings

4.1 Findings Summary

#	Finding Title	Category	Resource	MITRE	Phase	Severity	Evidence
1	Long-lived secrets in source control create persistent exposure	Credential Exposure	Git repository / IAM key pattern	T1552.001 / T1078	C/D	Critical	PhaseD_lifecycle.pdf
2	Secret detection tools may miss synthetic/unsupported patterns	Detection Coverage Gap	TruffleHog / repository scanning	T1552.001	C	Medium	trufflehog_output.txt
3	CloudGoat setup constrained by IAM permissions and CloudShell resources	Lab / Access Control Constraint	AWS IAM / CloudShell / CloudGoat	T1078 (context)	B	Low	phaseB_setup.png
4	Hosted-lab evidence not yet supplied	Evidence Completeness	PwnedLabs / TryHackMe	N/A	E	Info	phaseE_lab1.png / phaseE_lab2.png pending

4.2 Detailed Finding Analysis

Finding 1: Long-lived secrets in source control create persistent exposure

Field	Analysis
Type / Category	Credential Exposure
Cloud Service / Resource	Git repository / IAM access-key pattern
Description	A long-lived credential committed to a repository can persist in Git history and clones even after the working-tree file is removed.
How Identified	Lifecycle analysis of the synthetic repository scenario and task threat model.
MITRE ATT&CK Reference	T1552.001 Credentials in Files; T1078 Valid Accounts
Business / Security Impact	Confidentiality and integrity risk: a valid exposed key could allow unauthorised API access within its permissions until disabled or rotated.
Severity / Impact	Critical
Evidence Reference	phaseD_lifecycle.pdf

Finding 2: Secret detection tools may miss synthetic/unsupported patterns

Field	Analysis
Type / Category	Detection Coverage Gap
Cloud Service / Resource	TruffleHog v3.97.1
Description	The supplied TruffleHog run completed with 0 verified and 0 unverified secrets despite deliberately secret-like synthetic values.
How Identified	TruffleHog scan output and completion summary.
MITRE ATT&CK Reference	T1552.001 Credentials in Files
Business / Security Impact	Reliance on a single scanner can create false assurance. Multiple detectors plus manual review and pre-commit controls improve coverage.
Severity / Impact	Medium
Evidence Reference	trufflehog_output.txt

Finding 3: Lab setup constrained by permissions and storage

Field	Analysis
Type / Category	Access Control / Operational Constraint
Cloud Service / Resource	AWS IAM / CloudShell / CloudGoat
Description	The [REDACTED] identity was denied iam:ListUsers, and CloudShell storage exhaustion interrupted Terraform provider installation. CloudGoat also required explicit profile configuration.
How Identified	AWS CLI and CloudGoat terminal troubleshooting.
MITRE ATT&CK Reference	T1078 Valid Accounts (identity context)
Business / Security Impact	These constraints did not constitute a production vulnerability, but they affected reproducibility and demonstrated the importance of least privilege, correct profiles, and adequate lab resources.
Severity / Impact	Low
Evidence Reference	phaseB_setup.png

5.0 Risk Assessment & Cloud Threat Model

5.1 Cloud Security Control Gaps & Defensive Coverage

Finding / Gap	Observable / Trigger	Recommended Control	AWS Service / Tool
Repository secret exposure	Secret pattern appears in commit/PR or CI scan	Block commit/merge, revoke leaked credential, remove from history, rotate immediately	GitLeaks/TruffleHog + CI; AWS IAM / Secrets Manager
Long-lived IAM access key	Old active access key; anomalous use	Prefer roles/STS; least privilege; disable and rotate unused keys	IAM, CloudTrail, Access Analyzer
Hardcoded application secret	Secret stored in .env/config/source	Externalise secret and retrieve at runtime	AWS Secrets Manager or SSM Parameter Store + KMS
Scanner coverage gap	Different scanners return inconsistent results	Use layered scanning plus manual triage and validation	TruffleHog, GitLeaks, code review

5.2 Risk Exposure Over Time

Risk 1 - Long-lived repository credential: If a real key remains active after being committed, exposure persists through Git history, forks, caches, and developer clones. Rotation/revocation is therefore more important than simply deleting the visible file.

Risk 2 - Hardcoded automation/API token: Reuse across environments increases blast radius and makes ownership/rotation difficult. Moving the secret into a managed store and issuing scoped, short-lived credentials reduces persistence.

Risk 3 - Scanner coverage gap: A zero-finding result from one tool should not be treated as proof that no secret exists. Over time, undetected secrets can survive through releases unless pre-commit, CI scanning, and manual review are combined.

6.0 Recommendations

6.1 Technical Remediation

#	Finding	Specific Remediation	Priority	Reference
1	Long-lived secret in repository	Immediately deactivate/rotate the credential; purge it from repository history; replace hardcoded use with runtime retrieval from Secrets Manager/Parameter Store.	Immediate	NIST CSF PR.AC-1; ISO 27001 A.9
2	Single-scanner coverage gap	Run both GitLeaks and TruffleHog in CI/pre-commit workflows and manually validate findings; fail builds on confirmed high-confidence secrets.	Short-term	OWASP SAMM Implementation & Verification
3	Overly persistent credentials	Replace IAM user access keys where feasible with IAM roles and STS temporary credentials; apply least privilege and monitor CloudTrail use.	Short-term	NIST SP 800-53 AC-2; T1078
4	Weak lifecycle governance	Define owner, purpose, storage location, rotation interval, revocation process, and incident response for every non-human secret.	Long-term	CIS Control 6; ISO 27005

6.2 Strategic & Operational Improvements

- Adopt AWS Secrets Manager or SSM Parameter Store with KMS encryption for application secrets rather than repository storage.
- Prefer workload roles and STS temporary credentials over long-lived IAM user access keys.
- Integrate secret scanning at developer pre-commit, pull-request, and CI/CD stages and require human triage of findings.
- Create a documented emergency revocation workflow that treats a committed valid credential as compromised even if the commit is later deleted.
- Track secret ownership and rotation status so stale credentials can be identified and removed.

7.0 Reflection

Q1. How would you design an automated system to detect leaked credentials before code merges into production?

I would integrate secret-scanning tools such as GitLeaks and TruffleHog into both pre-commit checks and the CI/CD pipeline. Every pull request would automatically scan new commits for access keys, tokens, passwords, and other sensitive values before allowing a merge. High-confidence findings would block the pipeline and require remediation or security review. Secrets needed by applications would be stored in AWS Secrets Manager or Parameter Store instead of source code.

Q2. What business risks arise when developers use personal IAM keys instead of role-based temporary credentials?

Personal IAM access keys are typically long-lived, increasing the period in which a leaked credential can be abused. If a developer stores a key in source code or accidentally publishes it, an attacker may gain access to AWS resources using that developer's permissions. This could result in data exposure, unauthorized changes, financial losses, and compliance issues. Role-based temporary credentials reduce these risks by providing controlled, short-duration access.

Q3. How can short-lived tokens reduce attack surface compared to static API keys?

Short-lived tokens automatically expire after a limited period, whereas static API keys can remain valid until they are manually rotated or disabled. If a temporary credential is exposed, the attacker's opportunity to use it is therefore limited. Temporary credentials can also be issued through IAM roles with narrowly defined permissions. This reduces both the duration and potential impact of credential compromise.

Q4. If your organization discovered leaked secrets on GitHub today, what immediate containment steps would you recommend?

I would first treat the exposed credentials as compromised and immediately disable or revoke them. Replacement credentials should then be securely generated and affected systems updated without exposing the new values in source code. The organization should investigate AWS CloudTrail and related logs for unauthorized activity, determine the scope of exposure, and remove the secret from Git history and other copies. Finally, the root cause should be addressed by moving secrets to a managed service such as AWS Secrets Manager and implementing automated repository scanning to prevent recurrence

8.0 Deliverables Submission Checklist

Phase	Deliverable	Filename	Status
Phase A	Credential diagram + identity CLI screenshot	phaseA_identity.png	Complete
Phase B	CloudGoat setup / created-resource terminal screenshot	phaseB_setup.png	Complete
Phase C	TruffleHog output	trufflehog_output.txt	Complete
Phase C	GitLeaks JSON report	gitleaks_report.json	Complete
Phase C	AI summary screenshot	phaseC_summary.png	Complete
Phase D	Lifecycle diagram + matrix + recommendations	phaseD_lifecycle.pdf	Complete
Phase E	PwnedLabs Git Repos completion	phaseE_lab1.png	Pending
Phase E	PwnedLabs CodeCommit/Docker completion	phaseE_lab2.png	Pending
Phase E	TryHackMe Secure GitOps	phaseE_thm.png	Optional / Pending
Report	Final internship task report	CSE_CSE-AWS-L-013-04_SK_20260829.pdf	Complete

9.0 Additional Observations

CloudShell storage limitations materially affected the original SecLists workflow. Storage was recovered by clearing cache/Terraform provider cache and replacing the large target with a small synthetic repository. This preserved the learning objective while avoiding unnecessary resource use. The CloudGoat setup also showed that the active [REDACTED] identity lacked iam:ListUsers and that CloudGoat required an explicit AWS profile configuration.

Declaration & Sign-Off

I confirm that:

- All work in this report is my own except where AI usage is explicitly declared in Section 2.3.
- All technical activities were performed exclusively within the authorised cloud environments listed in Section 1.3.
- No external AWS accounts, production environments, third-party cloud resources, or real customer data were accessed during this task.
- All AWS credentials, access keys, session tokens, and account identifiers used during this task have been redacted in evidence files before submission.
- I have reviewed this report for accuracy, completeness, and professional presentation before submission.

Intern Full Name	Signature	Date of Submission
Sukhmandeep Kaur	Sukhmandeep Kaur	29 Aug 2026

EncryptEdge Labs Limited — Junior Cloud Security Engineer Internship Programme

Company No. 15830711 | United Kingdom | encryptedgelabs.com

Copyright © 2026 EncryptEdge Labs. All rights reserved. | Report Template v3.0